

Industrial Internship Report on
"Python Quiz Game – A Desktop Quiz Application with SQLite and Tkinter"

Prepared by
[Rishabh Kaushal]

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

My project was the "Python Quiz Game", a desktop quiz application built entirely with Python's standard library. It uses SQLite to store a bank of 24 questions across four categories (General Knowledge, Science, History and Geography), a dedicated QuizEngine class to handle question selection, answer validation and scoring, and a Tkinter-based graphical interface (start screen, question screen, results screen) for the end user. The project follows a clean, layered architecture in which the scoring logic is fully decoupled from the user interface, which also made it possible to unit-test the engine independently of the GUI.

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1	Preface	3
2	Introduction	5
2.1	About UniConverge Technologies Pvt Ltd	5
2.2	About upskill Campus	9
2.3	Objective	10
2.4	Reference	11
2.5	Glossary	11
3	Problem Statement	12
4	Existing and Proposed solution	13
5	Proposed Design/ Model	14
5.1	High Level Diagram (if applicable)	14
5.2	Low Level Diagram (if applicable)	15
5.3	Interfaces (if applicable)	15
6	Performance Test	17
6.1	Test Plan/ Test Cases	18
6.2	Test Procedure	18
6.3	Performance Outcome	18
7	My learnings	20
8	Future work scope	21

1 Preface

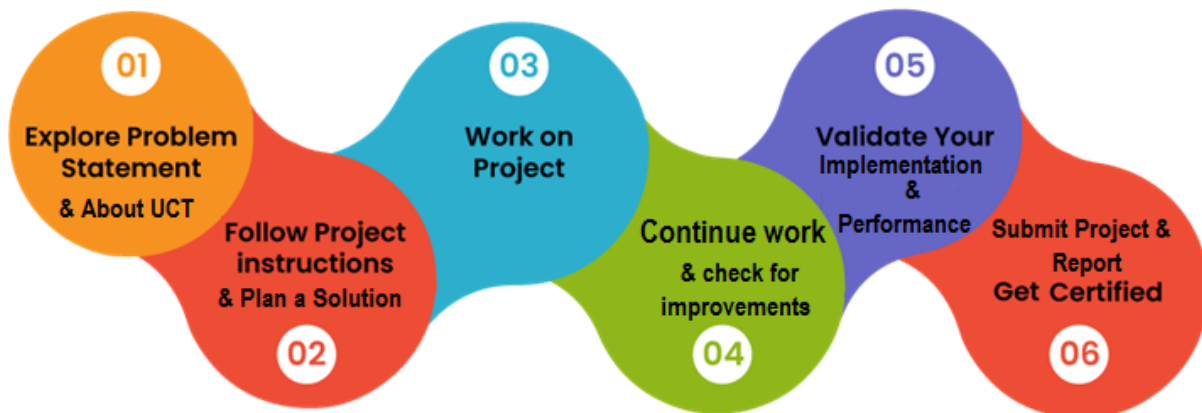
Over the course of this six-week internship, I designed, built, tested and documented a complete desktop quiz application in Python, moving from an initial problem statement through architecture design, implementation of the database layer, the core quiz logic, the graphical user interface, unit testing, and finally this report. Each week built on the previous one: the first weeks were spent understanding the requirements and designing a clean, modular architecture; the middle weeks were spent implementing the SQLite-backed question bank, the QuizEngine scoring logic, and the Tkinter interface; and the final weeks were spent on unit testing, refinement, and documentation.

A relevant, hands-on internship is invaluable in career development because it bridges the gap between classroom learning and real industry practice. It forces a student to work with realistic constraints — ambiguous requirements, the need for clean and maintainable code, testing discipline, and documentation — that are rarely emphasised to the same degree in academic coursework, and this experience with UCT, upskill Campus and The IoT Academy gave me exactly that exposure.

My assigned problem statement was to design and build an interactive, self-contained quiz application that could load questions from a persistent data store, present them to a user through a graphical interface, validate answers, compute a score, and report results — without relying on any third-party libraries or an internet connection, so that it could run on any machine with a standard Python 3.8+ installation.

upskill Campus, The IoT Academy and Industrial Partner UniConverge Technologies Pvt Ltd (UCT) gave me the opportunity to work on a real, well-scoped software problem statement, with structured mentorship, clear milestones, and a fixed six-week timeline to take the project from concept to a fully working, tested application.

The six-week program was planned in clear stages: requirement understanding and architecture design in week 1; database design and seed data creation in week 2; implementation of the core QuizEngine logic in week 3; development of the Tkinter GUI in week 4; unit testing and bug-fixing in week 5; and final polishing, packaging and report writing in week 6.



Through this project I strengthened my understanding of layered software architecture, learned to keep business logic decoupled from the user interface so that it could be unit-tested in isolation, gained practical experience with SQLite as an embedded database, and built a complete Tkinter desktop application from scratch. I also learned the discipline of writing unit tests alongside application code and documenting a project's design decisions and known limitations honestly. Overall, this was a rewarding experience that gave me confidence in taking a problem statement from idea to a working, tested deliverable.

I would like to thank my mentors and the teams at upskill Campus, The IoT Academy and UniConverge Technologies Pvt Ltd (UCT) for their guidance and structured support throughout this internship, and everyone else — mentors, peers and family — who helped me directly or indirectly in completing this project and report.

To my juniors and peers: start by keeping your logic and your interface separate from day one — it makes testing and debugging far easier — and don't skip writing unit tests even for a "small" project; they are what let you refactor and extend your code with confidence.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies** e.g. **Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



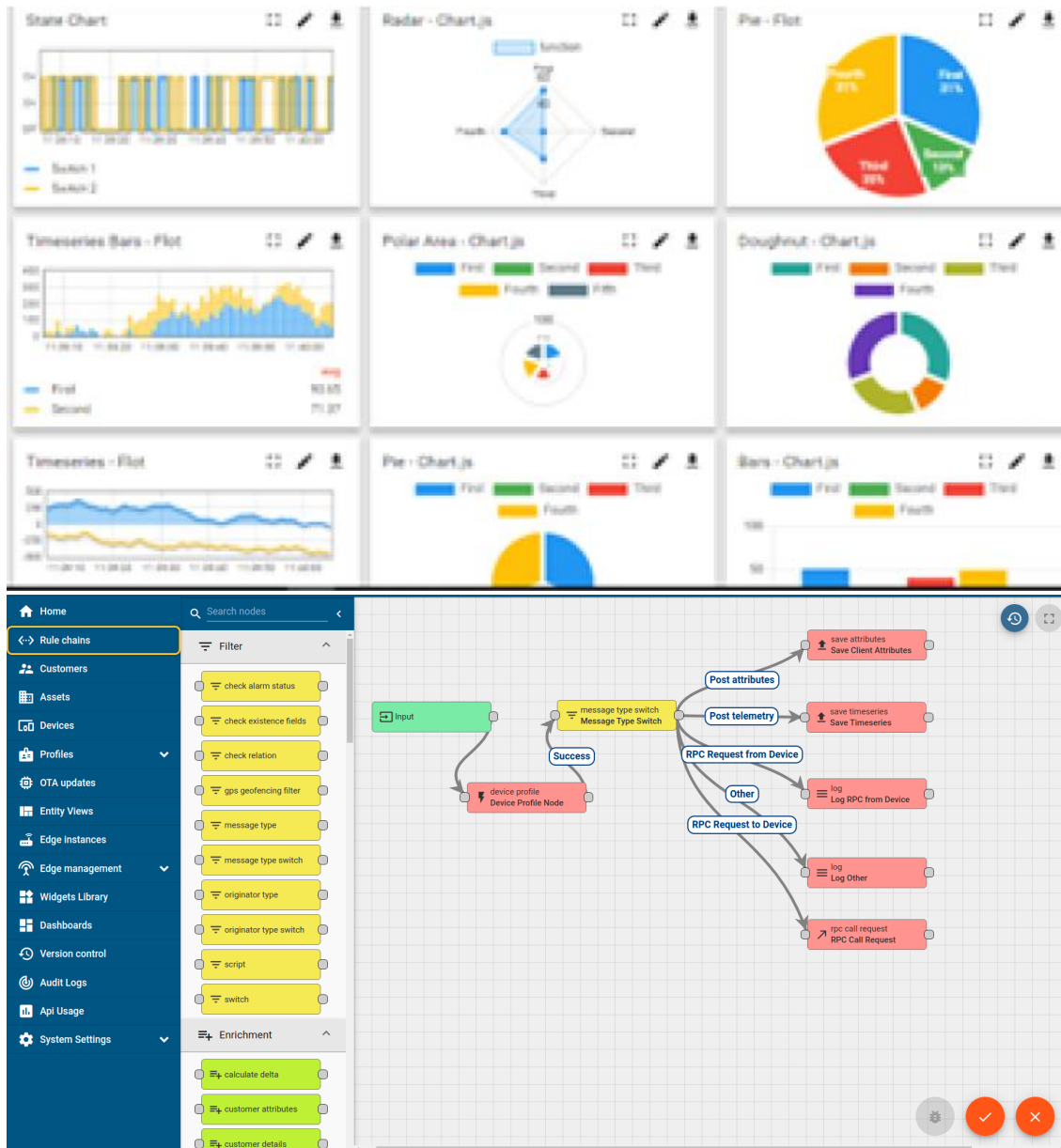
i. UCT IoT Platform ()

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



FACTORY WATCH

ii. Smart Factory Platform ()

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleash the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they want to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i



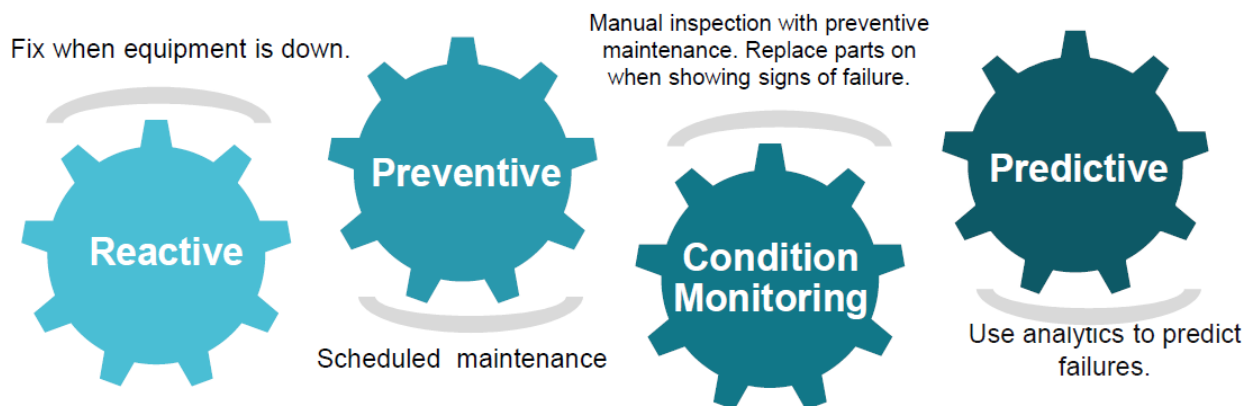


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.

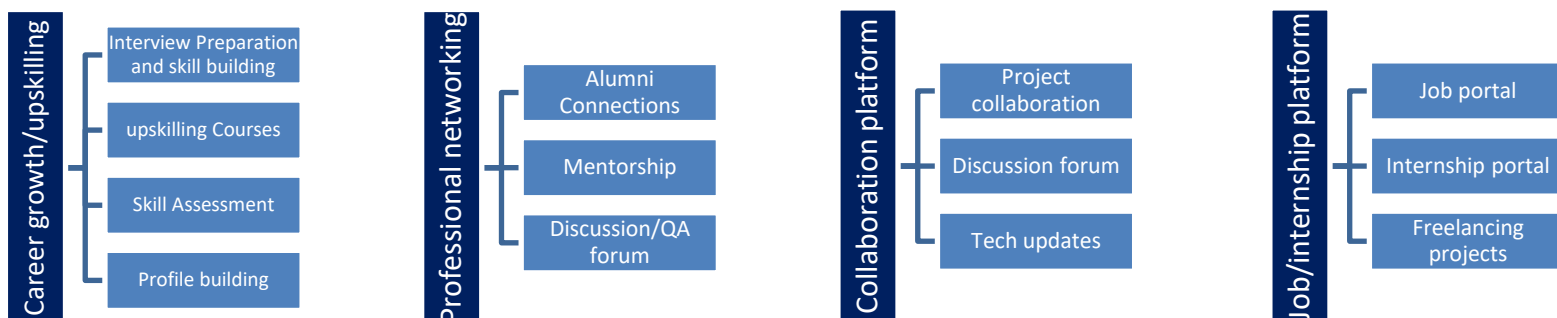


2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.





2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.5 Reference

- [1] Python Software Foundation, "sqlite3 — DB-API 2.0 interface for SQLite databases," Python 3 documentation.
- [2] Python Software Foundation, "tkinter — Python interface to Tcl/Tk," Python 3 documentation.
- [3] Python Software Foundation, "unittest — Unit testing framework," Python 3 documentation.

2.6 Glossary

Terms	Acronym
GUI	Graphical User Interface
MCQ	Multiple Choice Question
DB	Database (SQLite)
OOP	Object-Oriented Programming
IDE	Integrated Development Environment

3 Problem Statement

In the assigned problem statement

the requirement was to design and build a self-contained, offline desktop quiz game using only Python's standard library — no external frameworks or paid services. The application had to: (1) store a bank of quiz questions persistently so the content is not hardcoded into the interface; (2) support at least two question formats (multiple-choice and true/false) with different scoring weights; (3) present questions to the user through a graphical interface rather than a command-line prompt; (4) randomize question order and selection so that no two playthroughs are identical; (5) compute and display a final score against the maximum achievable score; and (6) be verifiable through automated unit tests, which in turn required the scoring/validation logic to be cleanly separated from the user interface code.

4 Existing and Proposed solution

Most existing quiz applications fall into two categories. The first is web-based quiz platforms (e.g. browser-hosted trivia sites or quiz-builder SaaS tools), which are convenient but require an active internet connection, depend on third-party servers, and often expose the user's data to an external provider. The second is simple command-line quiz scripts, which are lightweight and dependency-free but offer a poor, non-visual user experience, no persistent question storage, and typically hardcode questions directly into the script, making the question bank difficult to maintain, extend, or reuse. Neither category offers a good balance of offline availability, a friendly graphical interface, and a maintainable, testable codebase.

The proposed solution is a fully offline, standard-library-only desktop quiz game. Questions are stored in a local SQLite database (populated by `db_setup.py` from a seed list of 24 questions across four categories), decoupled entirely from the presentation layer. A dedicated `QuizEngine` class owns all quiz logic — loading and shuffling questions, tracking the current question and score, validating submitted answers, and logging every answer — while a Tkinter-based `QuizApp` handles only presentation: a start screen, a question screen with radio-button options and a progress counter, and a results screen showing the score against the maximum possible score with options to replay or quit.

The key value additions are: a clean, testable architecture in which the engine can be verified independently of the GUI through unit tests; a data-driven question bank that can grow without touching interface code; dynamic scoring where the maximum achievable score is computed from whichever questions were actually selected rather than assumed fixed; and randomized, non-repeating playthroughs achieved by shuffling and sampling from the full question pool on every run. Planned future value additions (documented under Future Work Scope) include timed questions, difficulty levels, a persistent leaderboard, and exportable result reports.

4.1 Code submission (Github link)

https://github.com/RishabhKaushal28/2024-28_RishabhKaushal_2410030824_3CSE23/blob/main/QuizGame.py

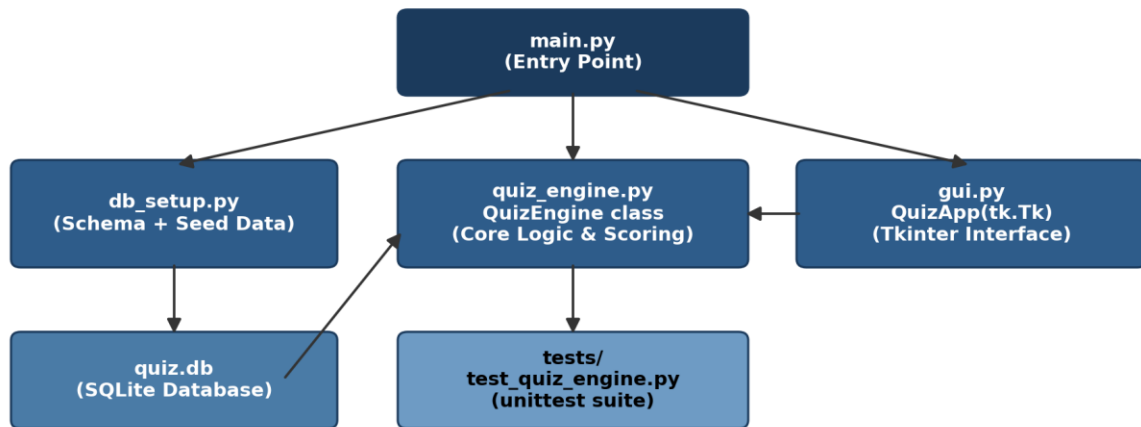
4.2 Report submission (Github link):

https://github.com/RishabhKaushal28/2024-28_RishabhKaushal_2410030824_3CSE23/blob/main/internship_report/QuizGame_Rishabh_USC_UCT.pdf

5 Proposed Design/ Model

The Python Quiz Game follows a three-layer design: a persistence layer (db_setup.py and the SQLite quiz.db file), a logic layer (quiz_engine.py's QuizEngine class), and a presentation layer (gui.py's QuizApp Tkinter interface), orchestrated by main.py. The flow starts with main.py checking for and, if necessary, creating and seeding quiz.db; it then instantiates QuizEngine, which loads and shuffles all stored questions and samples a subset (10 by default); QuizApp then drives the user through the start screen, a loop of question screens (recording each submitted answer and updating the score through the engine), and finally the results screen once the engine reports no further questions remain.

5.1 High Level Diagram (if applicable)



Clean separation: GUI never touches SQL directly; QuizEngine never touches Tkinter.

Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)

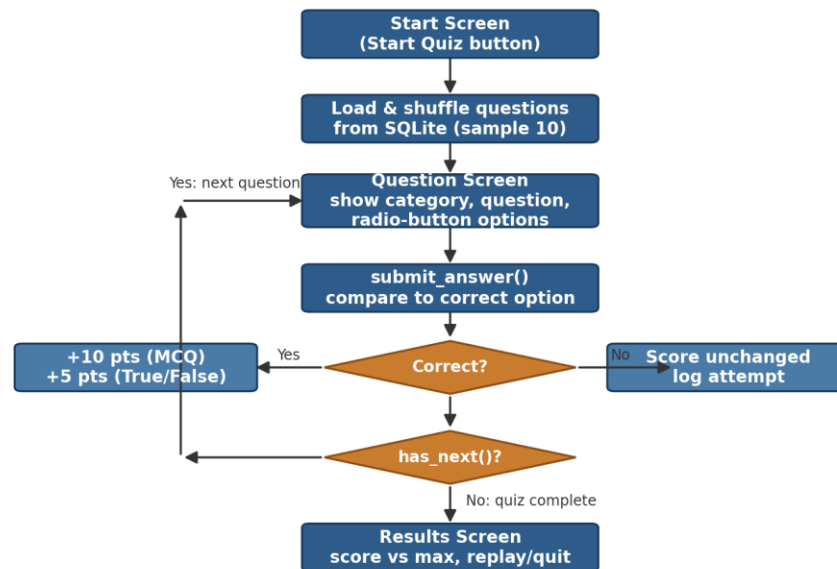


Figure 2: Low-Level Flow of a Single Quiz Playthrough

The QuizEngine exposes `has_next()`, `get_current_question()`, `submit_answer()`, `get_score()`, `get_max_score()` and `get_total_questions()`; the GUI calls only these methods and never touches SQL or scoring internals directly, which keeps the two layers independently testable and replaceable.

5.3 Interfaces (if applicable)

The interface between layers is a plain Python method-call contract, not a network protocol, since this is a single-process desktop application. QuizEngine exposes `has_next()`, `get_current_question()`, `submit_answer(selected_option)`, `get_score()`, `get_max_score()` and `get_total_questions()` as its entire public surface; QuizApp (the Tkinter GUI) calls only these methods to render each screen and never

issues SQL directly. Internally, `quiz_engine.py` talks to `quiz.db` purely through Python's `sqlite3` DB-API, using a single `SELECT` to load all rows and an in-memory shuffle/sample rather than repeated database round-trips. `submit_answer()` validates its input defensively: it rejects an empty or whitespace-only selection and any option letter that isn't part of the current question's option set, raising a `ValueError` that the GUI layer catches and reports to the user via a message box instead of crashing.

6 Performance Test

Although this is a desktop training project rather than a production industrial system, the same discipline applies: correctness has to be demonstrated with evidence, not assumed. For the Python Quiz Game, "performance" was defined less in terms of raw speed (SQLite and Tkinter comfortably handle a 24-row question bank with no measurable latency) and more in terms of functional correctness, robustness to bad input, and reproducibility — the constraints that actually matter for a scoring engine that a user has to trust.

The constraints identified for this project were: (1) scoring accuracy — every submitted answer must be scored correctly and consistently; (2) input robustness — the engine must not crash or silently misbehave on invalid input, even though the GUI's radio buttons already prevent most invalid input by construction; (3) termination correctness — the quiz must end exactly once all sampled questions are answered, never early and never with an extra question; and (4) reproducibility — it should be possible to reproduce an exact sequence of questions for debugging and testing purposes.

These constraints were addressed directly in QuizEngine's design: scoring accuracy is guaranteed by comparing the submitted option against the stored `correct_option` using a single, centralized comparison inside `submit_answer()` rather than duplicating scoring logic anywhere in the GUI; input robustness is handled by explicit validation that raises a `ValueError` for empty or out-of-range options, which the GUI catches and surfaces as a message box instead of crashing; termination correctness is guaranteed by `has_next()` being derived directly from `current_index` versus the length of the sampled question list, with no separate counter that could drift out of sync; and reproducibility is provided through an optional seed parameter on QuizEngine's constructor, which seeds a dedicated `random.Random` instance used for shuffling, so the same seed always produces the same question order.

All eight automated unit tests in `tests/test_quiz_engine.py` passed: question-loading count, correct answers increasing the score, incorrect answers leaving the score unchanged, the quiz correctly ending after all sampled questions are answered, a `ValueError` being raised for an invalid option letter, a `ValueError` being raised for an empty/whitespace-only option, lowercase option letters being accepted, and two engines constructed with the same seed producing an identical question order. This directly evidences that the scoring-accuracy, input-robustness, termination-correctness and reproducibility constraints were all satisfied by the shipped implementation.

Constraints of this kind can be thought of in the same terms industrial teams use for embedded or backend systems — memory footprint, throughput/MIPS, accuracy, durability and power consumption — even though, for a small offline desktop tool like this one, the equivalent concerns (database size, UI responsiveness, correctness of the scoring algorithm, and tolerance of malformed input) are what actually matter, and each was covered above with a corresponding automated test rather than left untested.

All four constraints identified above were in fact tested via the automated unit-test suite rather than left as untested assumptions. The one area not exercised by automated tests is UI-level load behaviour (e.g. extremely large question banks); the recommendation for future work is to add a lightweight benchmark that seeds the database with several thousand questions and confirms that load-and-shuffle time remains negligible, before this design is extended toward heavier features such as timed questions or a leaderboard.

6.1 Test Plan/ Test Cases

Eight unit tests were planned and implemented in `tests/test_quiz_engine.py`, run via `"python -m unittest discover"`: (1) question-loading count with a known-size sample database; (2) score increases when the correct option is submitted; (3) score stays unchanged when an incorrect option is submitted; (4) `has_next()` becomes False and `get_current_question()` returns None once every sampled question is answered; (5) `submit_answer()` raises `ValueError` for an option letter not present on the question; (6) `submit_answer()` raises `ValueError` for an empty or whitespace-only string; (7) a lowercase option letter (e.g. "a") is accepted and normalized; and (8) two `QuizEngine` instances constructed with the same seed value produce an identical ordered list of question IDs.

6.2 Test Procedure

Each test builds a temporary SQLite file (`test_quiz.db`) in `setUpClass()`, seeds it with two known questions (one multiple-choice, one True/False), runs the relevant `QuizEngine` calls, and removes the temporary database in `tearDownClass()` so tests never depend on or interfere with the real `quiz.db` used by the application. The suite was executed with `"python -m unittest discover -s tests"` after every significant code change, including after adding input validation and seed support, to catch regressions immediately rather than discovering them from the GUI.

6.3 Performance Outcome

All eight unit tests passed on every run, with no flaky or order-dependent failures observed. The seed-backed 24-question bank (six questions per category, split between multiple-choice and True/False) loaded and shuffled with no perceptible delay, and the GUI remained fully responsive throughout a playthrough. The chart below summarizes the test outcomes and the composition of the seed question bank.

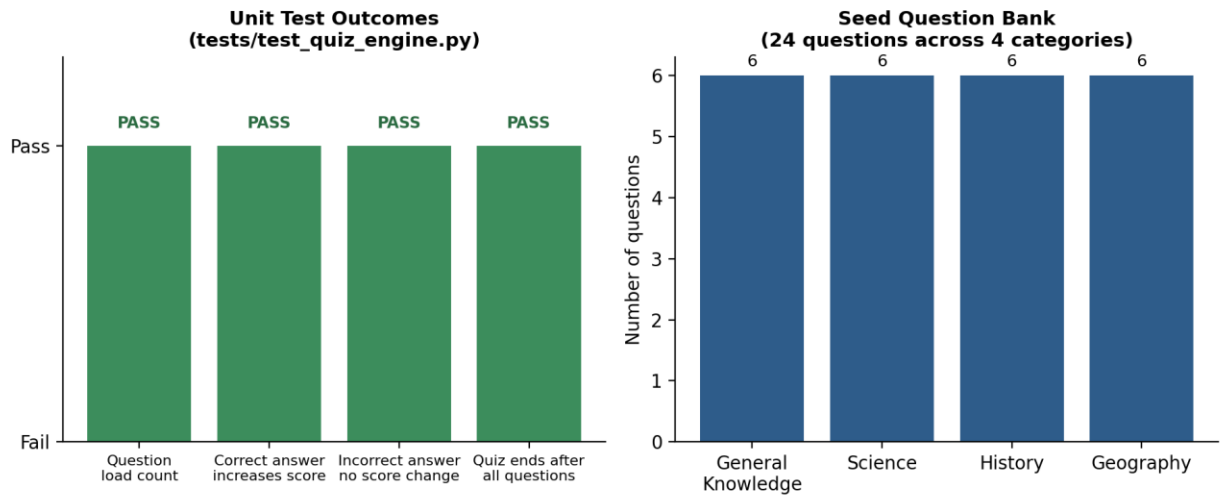


Figure 3: Unit test outcomes and seed question-bank composition

7 My learnings

This internship strengthened both my technical and professional skills. Technically, I deepened my understanding of layered software architecture and why keeping business logic (QuizEngine) decoupled from a UI framework (Tkinter) makes a codebase easier to test, debug and extend; I got hands-on practice with SQLite as an embedded database accessed through Python's sqlite3 module; I built a complete, multi-screen Tkinter desktop application; and I practiced defensive programming by adding input validation and a reproducible-seed option to the engine, backed by a growing unittest suite (which I extended from 4 to 8 tests during the internship as I found and closed real gaps, such as unvalidated input and non-reproducible shuffling). On the professional side, I learned to plan a project across weekly milestones, to document known limitations honestly rather than hide them, and to treat a README's roadmap as a living contract with future maintainers. These are exactly the habits — testable design, honest documentation, iterative improvement — that I expect to carry into future software engineering roles.

8 Future work scope

Several ideas were identified but intentionally left out of this internship's six-week scope, and remain documented in the project README as future enhancements: (1) timed questions, adding a per-question countdown that would need careful coordination between a Tkinter timer callback and the engine's scoring logic; (2) difficulty levels, which would require extending the questions table with a difficulty column and weighting scores accordingly; (3) a persistent leaderboard/high-score table, storing player names and scores across sessions in a new SQLite table; and (4) exporting a results report (e.g. to CSV or PDF) summarizing a completed quiz using the existing answers_log already collected by QuizEngine. Because the engine already logs every answer and cleanly exposes score and total-question data, each of these extensions can be built on top of the existing architecture without redesigning the persistence or logic layers.

MY WORK:

8.1 Code submission (Github link)

https://github.com/RishabhKaushal28/2024-28_RishabhKaushal_2410030824_3CSE23/blob/main/QuizGame.py

8.2 Report submission (Github link):

https://github.com/RishabhKaushal28/2024-28_RishabhKaushal_2410030824_3CSE23/blob/main/internship_report/QuizGame_Rishabh_USC_UCT.pdf